YogaSync AI — Master Engineering Manual

Complete Build, Architecture, Setup Commands & Troubleshooting Guide (Steps 1 to 6)

Project: YogaSync AI CV & Dataset Engine	**Target Repository:** yogasynca/CV-MODULE.git
Author / Lead: MAHAMMMAD THANSEEF	**Target Branch:** main (Synchronized)
Core Stack: Python 3.14, MediaPipe, OpenCV, Next.js 16, Supabase, Google Drive API	**Status:** Steps 1-6 Complete & Production-Verified

1. Executive Architecture Overview

YogaSync AI is a high-precision Computer Vision and AI posture assessment platform. It combines a Python-based MediaPipe landmark analysis engine with a secure Next.js Neo-Brutalist web application. The platform manages dataset collection, multi-role review workflows (Team Members, Reviewers, Project Mentor), cloud synchronization (Supabase PostgreSQL + Google Drive 400 GB storage), and real-time biomechanical angle computation for 8 canonical yoga asanas.

2. Step 1: Environment Setup & Dependencies Installation

This step initializes the isolated Python virtual environment and the Next.js Node.js project environment.

Step / Objective	Terminal Command (Windows PowerShell)	Function / What it Does
1.1 Create Python Virtual Environment	`python -m venv .venv`	Creates an isolated Python runtime in .venv/ avoiding system package pollution.
1.2 Activate Environment	`.venv\Scripts\Activate.ps1`	Activates the Python virtual environment for the current PowerShell session.
1.3 Install CV & AI Libraries	`.venv\Scripts\pip install mediapipe opencv-python numpy pillow reportlab`	Installs Google MediaPipe (33 body landmarks), OpenCV (image processing), NumPy (trigonometry/matrix math), and Pillow.
1.4 Initialize Next.js Portal	`cd portal` `npm install`	Installs Node.js dependencies: Next.js 16, React 19, Lucide-React icons, Supabase client SDK, Google APIs, Tailwind CSS.
1.5 Download MediaPipe Task Model	`Invoke-WebRequest -Uri "https://storage.googleapis.com/mediapipe-models/pose_landmarker/pose_landmarker_lite/float16/1/pose_landmarker_lite.task" -OutFile "pose_landmarker_lite.task"`	Downloads the official MediaPipe Lite pose landmark detection model weights (2.8MB).

3. Step 2: Computer Vision Core & Biomechanical Angle Calculator

The CV engine extracts 33 standard 3D skeletal landmarks and calculates joint angles across body planes using the law of cosines.

File Path	Key Classes / Functions	Biomechanical Role & Operation

`pose/angle_calculator.py`	`calculate_angle(a, b, c)` `calculate_all_angles(...)`	Calculates interior angles between joint triplets (e.g. Shoulder-Elbow-Wrist, Hip-Knee-Ankle) using vector dot products: $\theta = arccos((BA \cdot BC)/(\|BA\|\|BC\|))$.

`pose/landmark_extractor.py`	`PoseLandmarkExtractor`
	`extract_from_image()`
	`extract_from_video()`
		Feeds RGB frames into MediaPipe Pose Landmarker Task. Normalizes (x, y, z, visibility) coordinates and validates 33 keypoints.

`pose_detection.py`	`CLI Pipeline Handler` `draw_landmarks()`	Standalone CLI utility for live webcam feed or image file landmark overlay, rendering real-time angles on joint vertices.

4. Step 3: Dataset Ingestion & AI Data Pipeline

The dataset pipeline structures raw images/videos, runs automated quality gates, and outputs clean normalized JSONL/manifest artifacts.

File Path	Operations & Data Flow
`ai/dataset/collector.py`	Handles raw multi-format ingestion (JPEG, PNG, MP4, MOV). Assigns deterministic sample UUIDs, organizes folders by asana label.
`ai/dataset/extractor.py`	Batch processes raw media through PoseLandmarkExtractor, computing 12 critical joint angles and writing structured records to ai/data/processed/samples.jsonl.
`ai/dataset/validator.py`	Enforces 5-point quality gate: (1) Resolution ≥ 640x480, (2) Visibility score > 0.65 for key joints, (3) Boundary containment, (4) No extreme occlusion, (5) Anatomical plausibility.
`ai/dataset/metadata.py`	Generates and syncs ai/data/metadata/dataset_manifest.json containing total count, class distribution, camera angles, and split metadata.
`ai/dataset/seed_demo.py`	Generates validated synthetic demo samples for all 8 asanas to facilitate instant local development and UI testing.

5. Step 4: Database Schema & Cloud Storage Architecture

Supabase PostgreSQL provides relational state tracking and role-based permissions, while Google Drive provides 400 GB high-capacity media storage.

PostgreSQL Schema (supabase/schema.sql): Configures users, poses (8 canonical asanas with angle tolerance ranges), samples (upload records, status, GDrive URL), and reviews (approvals/rejections).

Row Level Security (RLS): Ensures Team Members can only modify their own submissions, Reviewers can update review states, and Mentors have full administrative auditing rights.

Google Drive OAuth Connector (portal/src/lib/storage/google-drive.ts): Implements chunked multipart uploads, automated token refreshing using OAuth Refresh Tokens, and folder hierarchy segregation.

6. Step 5: Neo-Brutalist Web Portal & Gated Authentication

The web frontend delivers a responsive Neo-Brutalist interface with mandatory authentication guarding all internal routes.

Component / Route	Path	Functionality & Behavior
Login Gate & Welcome	`src/app/page.tsx`	Mandatory login screen with username & password validation. Removed all 1-click bypass buttons. Unauthenticated users cannot enter.

Authentication Guard	`src/components/auth/AuthGuard.tsx`	Wraps all internal routes (/dashboard, /upload, /review, /admin). Redirects unauthenticated visitors to login immediately.
Authentication Context	`src/lib/auth-context.tsx`	Maintains session state, role-based redirection, Supabase authentication integration, and official credential matching.
Responsive Navbar	`src/components/layout/Navbar.tsx`	Neo-brutalist header featuring 3-line animated hamburger menu, brand badge, and user session badge with secure logout.
Upload Station	`src/app/upload/page.tsx`	Drag-and-drop file uploader with live client-side MediaPipe landmark preview, metadata capture, and GDrive transmission.
Reviewer Queue	`src/app/review/page.tsx`	Interactive inspection interface for Reviewers to approve/reject submissions with landmark overlay and angle checks.
Mentor Console	`src/app/admin/page.tsx`	Administrative dashboard with dataset balance charts, team productivity metrics, and global database purge/export tools.

7. Step 6: Testing, Build Verification & Git Deployment

Full verification suite ensuring all 20 Python unit tests pass, Next.js builds with 0 errors, and the repository is securely synchronized with GitHub.

Step / Verification	Command Executed	Result & Validation
6.1 Python Unit Tests	`.venv\Scripts\python.exe -m unittest discover -s tests -v`	20/20 unit tests passed (angle calculation, boundary normalization, metadata schema validation).
6.2 Next.js Production Build	`cd portal` `npm run build`	17/17 routes compiled successfully with zero TypeScript and zero ESLint errors.

6.3 Git Tracking Initialization	`git init`
	`git branch -M main`
	`git add .`
		Clean repository initialized in c:\Users\DELL\YogaSyncai with all 74 project files staged.

6.4 Remote Push to GitHub	`git remote add origin https://github.com/yogasynca/CV-MODULE.git` `git push -u origin main`	Successfully synchronized 15,736 lines of code to remote. PAT token sanitized from git config.

8. Complete Troubleshooting & Error Resolution Matrix

Detailed record of every technical challenge encountered during construction and the exact engineering fix applied:

Encountered Issue / Error	Root Cause Analysis	Exact Resolution Applied
Parent Directory Git Collision Git was initialized in C:\Users\DELL	Git tracking was tracking user root profile rather than the isolated project directory.	Cleaned parent git config, initialized dedicated git in YogaSyncai, created .gitignore excluding node_modules, .venv, .env.local.
Next.js Suspense Boundary Warning useSearchParams() outside Suspense	Next.js 16 requires client-side search query readers to be wrapped in Suspense to prevent server-side bailout.	Extracted PoseParamSync into an isolated subcomponent wrapped in <Suspense fallback=...> inside upload/page.tsx.
MediaPipe WASM / Task File 404 Cannot locate pose_landmarker.task	Client-side browser detector could not access local filesystem paths directly.	Copied pose_landmarker_lite.task into portal/public/ and configured static CDN fallback.

Git Remote Push Authentication Remote rejected push without token	GitHub disabled password authentication for Git CLI operations.	Used Personal Access Token (PAT) for push authentication, then sanitized remote URL to standard HTTPS.
OpenCV Headless vs GUI cv2.imshow() crashes on server	CI/Server environments lack display window managers.	Structured pose_detection.py to conditionally output rendered image files rather than blocking GUI windows when running non-interactively.

9. Role-Based Access Control & Security Architecture

The portal enforces strict role-based access control (RBAC). All internal routes are shielded behind the AuthGuard component, and API endpoints validate user permissions before granting write or review access:

Role Level	Authorized Capabilities	Target Route	Security Boundaries
Team Member / Contributor	Upload raw media (images/videos), capture pose metadata, execute client-side landmark validation.	`/upload`	Cannot approve/reject peer submissions or modify system configuration.
Reviewer / Quality Auditor	Inspect submitted samples, evaluate anatomical angles against tolerance thresholds, Approve/Reject records.	`/review`	Audit trail logged for all reviews; cannot purge global dataset.
Project Mentor / Admin	Full supervisory oversight, monitor dataset health & balance metrics, export dataset manifests, purge test records.	`/admin`	Elevated privileges with Supabase Service Role security integration.

10. Supported 8 Canonical Asanas & Biomechanical Targets

Asana Name	Sanskrit Name	Primary Target Angles	Critical Checkpoints
Mountain Pose	Tadasana	Spine 180°, Knees 180°, Arms 0°/180°	Vertical spine alignment, feet grounded
Tree Pose	Vrikshasana	Standing Knee 180°, Bent Knee 45°-60°	Foot against inner thigh, hands in Anjali Mudra
Warrior I	Virabhadrasana I	Front Knee 90°, Back Knee 180°, Arms 180°	Hips squared forward, chest lifted
Warrior II	Virabhadrasana II	Front Knee 90°, Back Knee 180°, Arms 180°	Arms parallel to floor, gaze over front fingers
Downward Dog	Adho Mukha Svanasana	Hips 70°-80°, Arms 180°, Knees 180°	Inverted V-shape, spine straight, heels reaching
Cobra Pose	Bhujangasana	Elbows 150°-170°, Chest-Hip 130°-150°	Pubic bone on floor, shoulders rolled back
Triangle Pose	Trikonasana	Front Knee 180°, Torso-Leg 90°, Arms 180°	Lateral flexion without collapsing chest forward
Bridge Pose	Setu Bandhasana	Knees 90°, Hip-Spine 160°-175°	Pelvis lifted high, thighs parallel